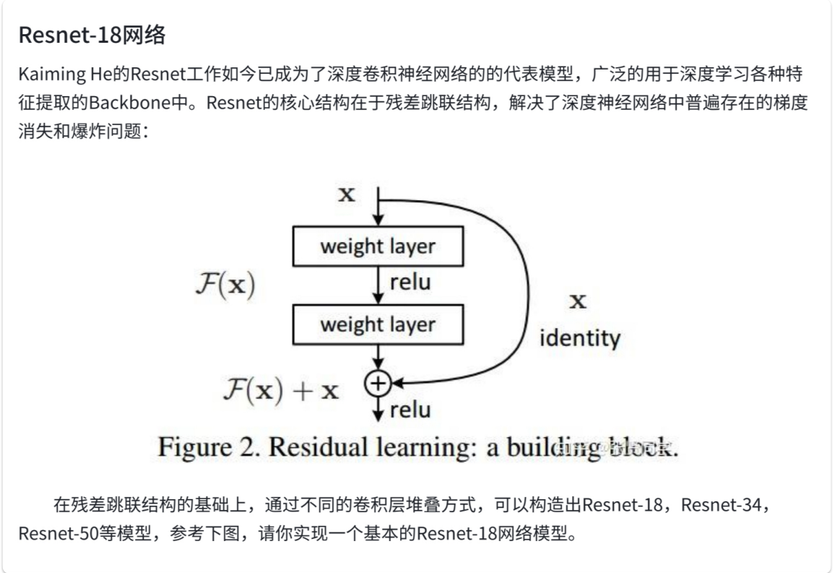
Dian 团队招新

一、ResNet 18网络

9/15～19 从零开始研究 ResNet-18 网络



Resnet-18网络

Kaiming He的Resnet工作如今已成为了深度卷积神经网络的的代表模型，广泛的用于深度学习各种特征提取的Backbone中。Resnet的核心结构在于残差跳联结构，解决了深度神经网络中普遍存在的梯度消失和爆炸问题：

x
weight layer
relu
weight layer
$\mathcal{F}(\mathbf{x})$
x
identity
$\mathcal{F}(\mathbf{x}) + \mathbf{x}$
relu

Figure 2. Residual learning: a building block.

在残差跳联结构的基础上，通过不同的卷积层堆叠方式，可以构造出Resnet-18，Resnet-34，Resnet-50等模型，参考下图，请你实现一个基本的Resnet-18网络模型。

什么是 ResNet 网络，有什么作用，背后原理是什么

Residual Networks（残差网络）

作用：通过残差跳联，将原先输入的数据处理为更好、更优质的数据

例：一般情况下，如果将一张低分辨率的图片放入一个训练过的神经网络中，未必得到更高分辨率的照片（数据会在训练过程中逐渐丢失）

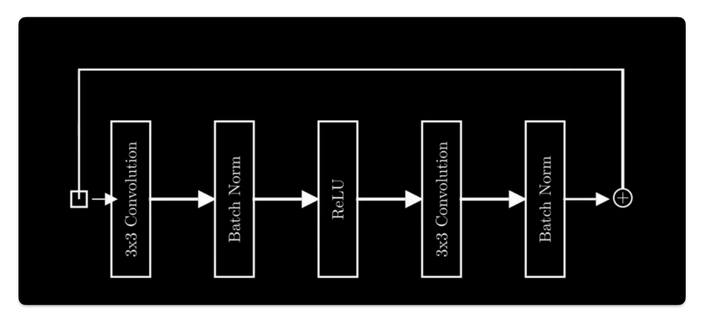
而resnet使输出的图像具有更高的分辨率。

原理：通过对于原始数据的处理，得到残差块，再将残差块与输入数据叠加，将数据进行初步处理，再将数据通过层层卷积，完成ResNet 18的搭建

什么是残差跳联？

答：我觉得可以理解为更好的输出结果与输入结果的差值。

也就是说，通过输入与这一残差的叠加，得到了更好的结果，而这些结果也可以继续与残差块叠加，来迭代训练更深层的网络。

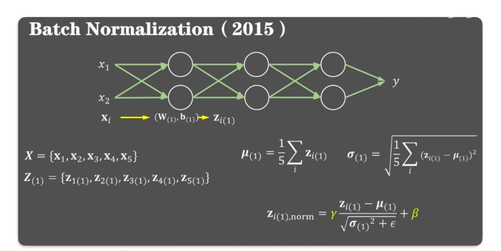


3x3 Convolution
Batch Norm
ReLU
3x3 Convolution
Batch Norm

生成残差块的流程：先将原始数据通过卷积神经网络处理，提取关键特征，再通过批量归一化，加速网络的收敛，然后通过激活函数（如ReLU）来对数据进行拟合，多次重复此步骤，最后将初始数据与输出数据叠加，形成一次残差跳联

什么是批量归一化，为什么要用bn算法？

答：在训练神经网络时，如果数据间的差异很大，提取出来的数据的特征就会不明显（或者收敛速度很慢，效果也不好），因此，要让数据尽量类似地输入，来确保网络收敛的质量与速度。批量归一化就是缩小数据差异的一项方法。



Batch Normalization（2015）
$X = \{x_1, x_2, x_3, x_4, x_5\}$
$\mu_{(1)} = \frac{1}{5}\sum z_{(1)}$
$z_{(1),norm} = \gamma\frac{z_{(1)}-\mu_{(1)}}{\sqrt{\sigma_{(1)}^2+\epsilon}}+\beta$

BN算法具体的过程

怎么理解卷积神经网络，在ResNet18中有什么作用

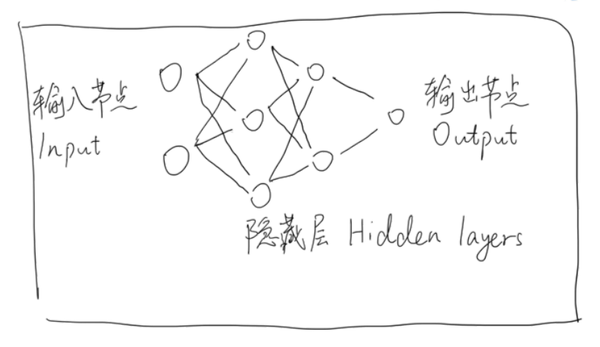
我认为卷积神经网络的目标是将数据的关键特征提取出来，生成更新数据；并控制维度与通道数。

一方面，使用卷积神经网络是为了生成特征更为明显的数据；另一方面，对于残差块，需要输入与输出的数据类型与维度一致，通过设定卷积核的大小和步长，使得输入数据与输出数据的维度一致，这样才能进行叠加。最后对此进一步进行卷积，构成ResNet18网络。

9/20～21 研究神经网络及ReLU激活函数

什么是神经网络？

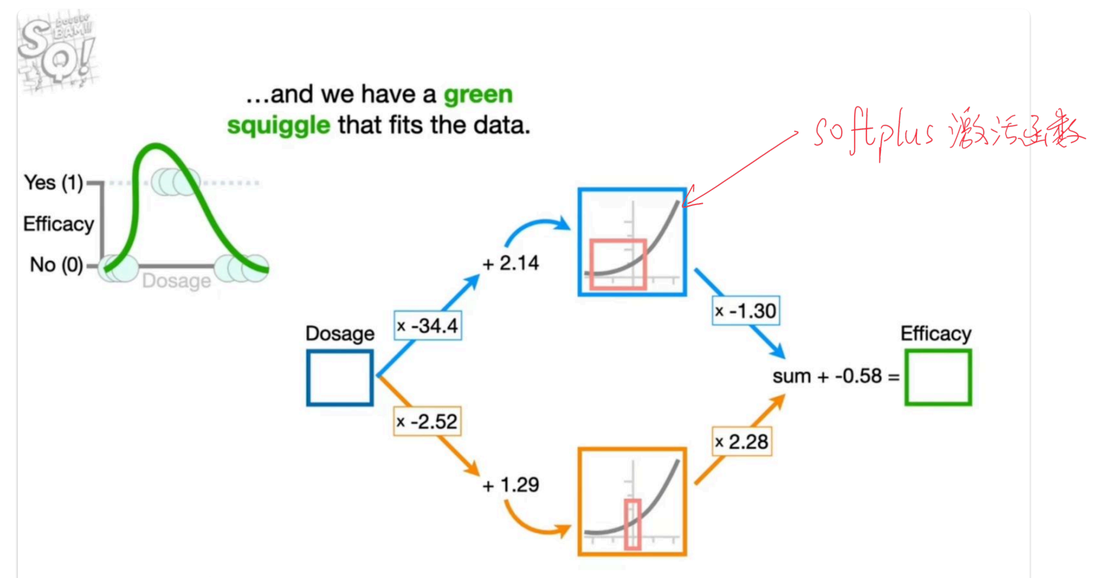
我认为神经网络是通过一系列计算形成的曲线拟合器，通过调整一系列参数以及其中采用的激活函数，来达到构建一套用于拟合数据的算法。



输入节点
Input
输出节点
Output
隐藏层 Hidden layers

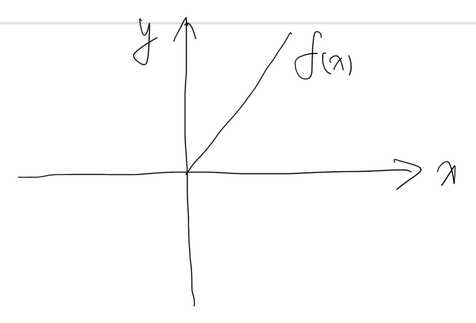
神经网络概念图

在一个神经网络中，激活函数是用来拟合图像的重要部分，我认为，之所以称其为激活函数，正是由于其能够使拟合的数据拟合成网络的一部分，就像是激活了这些数据一样。



...and we have a green squiggle that fits the data.
Yes (1)
Efficacy
No (0)
Dosage
x -34.4
+ 2.14
x -2.52
+ 1.29
x -1.30
x 2.28
sum + -0.58 =
Efficacy

softplus 激活函数



$$f(x)=\begin{cases}x, & x\geq 0\\ 0, & x<0\end{cases}$$

ReLU激活函数

9/22 SoftMax和ArgMax函数

背景：当神经网络有多个输出结果时，其结果的取值范围往往过于宽泛（可能大于1，也可能是负值，还可能是0），导致其意义难以理解，也很难直观地在输出结果间形成对比。因此，可以采用适当的函数对于输出数据进行处理。

ArgMax函数：将输出结果全部设置为0或1.

优势：输出结果简单易懂，直观性强

劣势：但同不能将输出结果之间的关系阐明，并且由于将值设为0，导致使用反向传播时，导数值总为0，反向传播就失效了

SoftMax函数：$SoftMax_{a_i}(Output\ Values)=\frac{e^{a_i}}{e^{a_1}+e^{a_2}+\dots+e^{a_n}}\quad(i=1,2,\dots,n)$

可以观察到 $\sum_{i=1}^{n} SoftMax_{a_i}(Output\ Values)=1$.

那么输出的结果全部集中于0到1的区间内，并且这些结果的总和等于1

也就是说，SoftMax输出值可以被解释为预测概率，使每一个输出值在整体中的权重更明显

并且，这些处理过的数据也可以用来做反向传播，更新参数

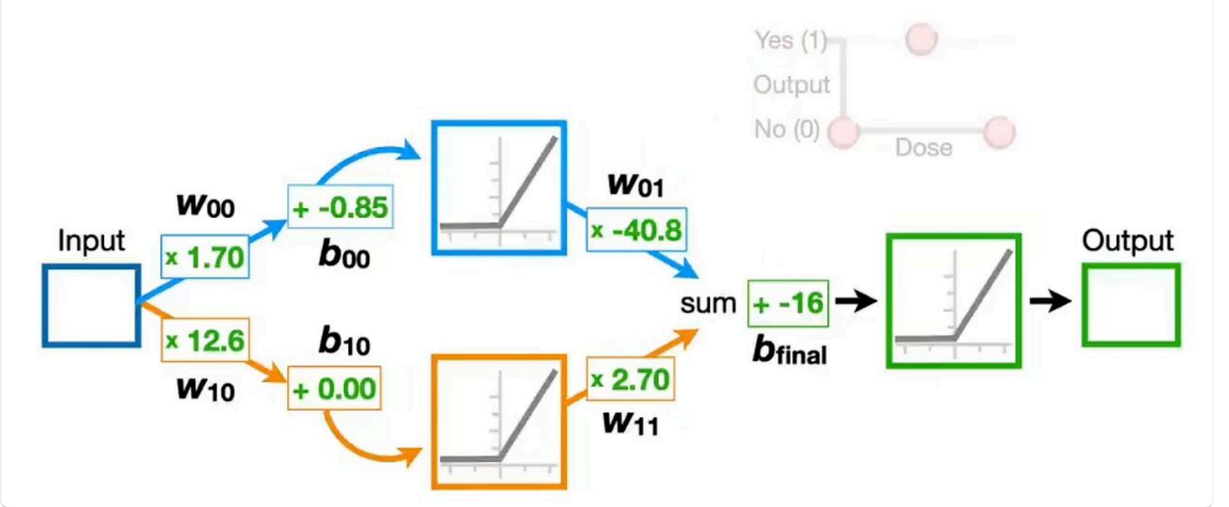
9/23 学习用PyTorch构建神经网络

一、导入Python模块

```
import torch                          #使用torch创建张量来存储数值
import torch.nn as nn
import torch.nn.functional as F  # 提供激活函数
from torch.optim import SGD   #用SGD(Stochastic Gradient Descent 随机梯度下降)让神经网络拟合数据

import matplotlib.pyplot as plt
import seaborn as sns
```

二、构建神经网络

```
class BasicNN(nn.Module):   # BasicNN 将继承自PyTorch的名为module的类
    def __init__(self):     # 创建初始化方法
        super().__init__()  # 调用父类的初始化方法
```

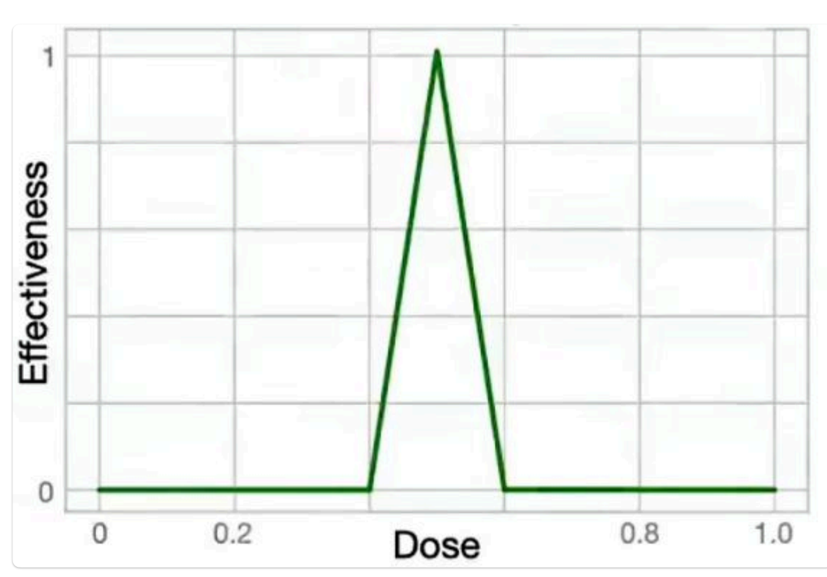


Input
w00
x 1.70
+ -0.85
b00
w10
x 12.6
b10
+ 0.00
w01
x -40.8
x 2.70
w11
sum
+ -16
bfinal
Output
Yes (1)
Output
No (0)
Dose

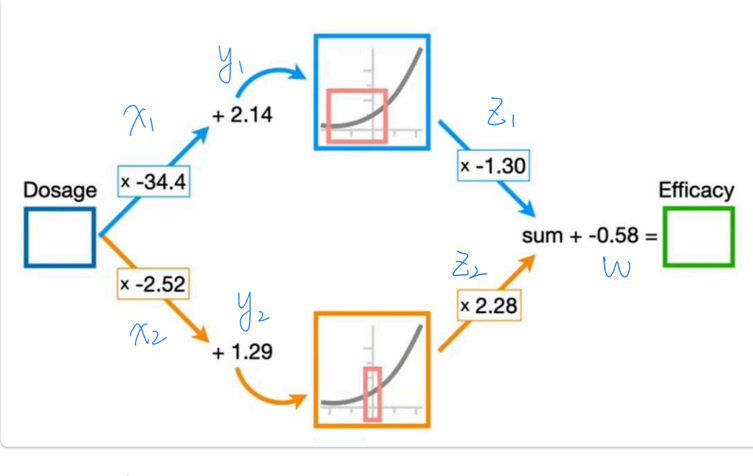
```
        self.w00 = nn.Parameter(torch.tensor(1.7), requires_grad=False)
        self.b00 = nn.Parameter(torch.tensor(-0.85), requires_grad=False)
        self.w01 = nn.Parameter(torch.tensor(-40.8), requires_grad=False)

        self.w10 = nn.Parameter(torch.tensor(12.6), requires_grad=False)
        self.b10 = nn.Parameter(torch.tensor(0), requires_grad=False)
        self.w11 = nn.Parameter(torch.tensor(2.7), requires_grad=False)

        self.bfinal = nn.Parameter(torch.tensor(-16), requires_grad=False)
    def forward(self, input):  # 使数据向前传递
        input_to_top_relu = input * self.w00 + self.b00
        top_relu_output = F.relu(input_to_top_relu)
        scaled_top_relu_output = top_relu_output * self.w01

        input_to_bottom_relu = input * self.w10 + self.b10
        bottom_relu_output = F.relu(input_to_bottom_relu)
        scaled_bottom_relu_output = bottom_relu_output * self.w11

        input_to_final_relu = scaled_top_relu_output + scaled_bottom_relu_output + self.bfinal

        output = F.relu(input_to_final_relu)

        return output
```

三、输入数据检验拟合图像是否符合训练集

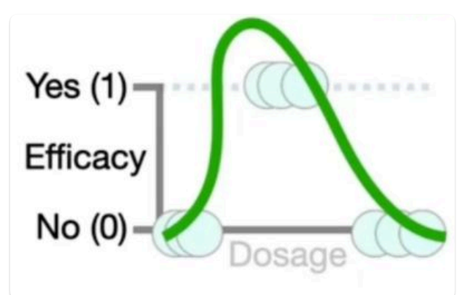
```
input_doses = torch.linspace(start=0, end=1, steps=11)  #创建 从0到1,步长为0.1的张量
# print(input_doses)
model = BasicNN()
output_values = model(input_doses)

sns.set(style="whitegrid")
sns.lineplot(x=input_doses,
             y=output_values,
             color='green',
             linewidth=2.5)
plt.ylabel('Effectiveness')
plt.xlabel('Dose')
plt.show()
```



如何确定合适的参数，来构建神经网络呢？



Dosage
x -34.4
x1
y1
+ 2.14
z1
x -1.30
x -2.52
x2
y2
+ 1.29
z2
x 2.28
sum + -0.58 =
w
Efficacy

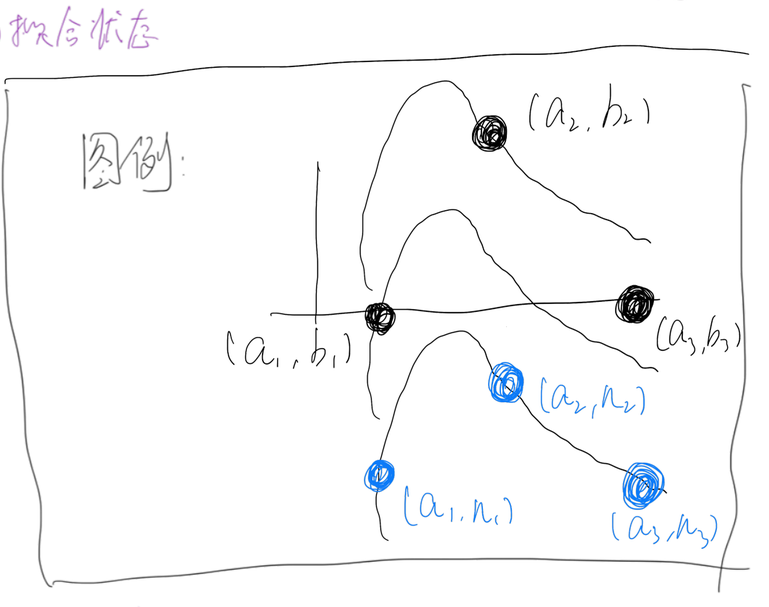


对于算法中的每个参数，都可以利用反向传播的思路来解决问题。

以上面两张图为例：左图是神经网络的具体运算过程，右图是最好的拟合结果

假设参数 $x_1, x_2, y_1, y_2, z_1, z_2$ 都是已知的，并且已经选定某种激活函数。

那么，此时拟合的曲线的形态已经可以确定，需要通过调整曲线的高度来使得达到最佳的拟合状态



图例：
(a_2, b_2)
(a_1, b_1)
(a_3, b_3)
(a_2, n_2)
(a_1, n_1)
(a_3, n_3)

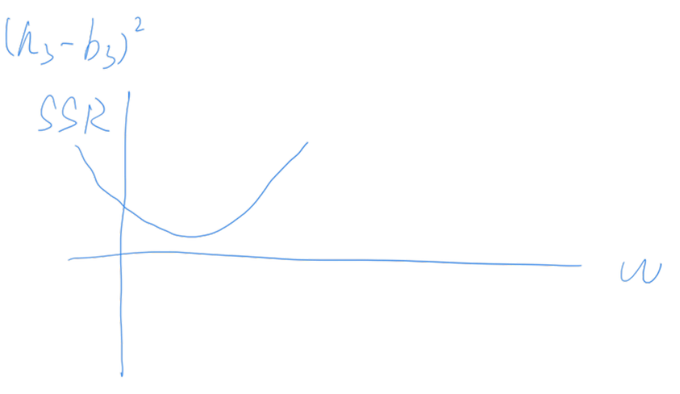
如上图，通过改变参数w来改变曲线的拟合程度，此时需要利用梯度下降的方法来找到局部最优解（我不知道怎么找到全局最优解）

同理，其它的参数也可以通过反向传播来计算，从而构建出一套完整的算法。

具体算法：计算残差平方和 SSR

$SSR=(n_1-b_1)^2+(n_2-b_2)^2+(n_3-b_3)^2$

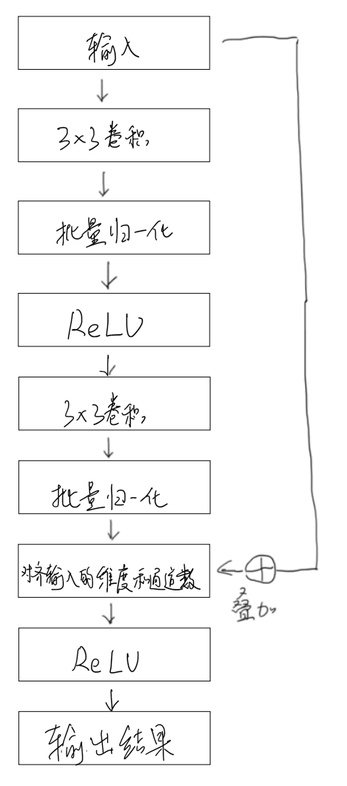
可构建 w 与 SSR 的函数



通过链式法则求导，计算导数值为0（或最小）时，w的值

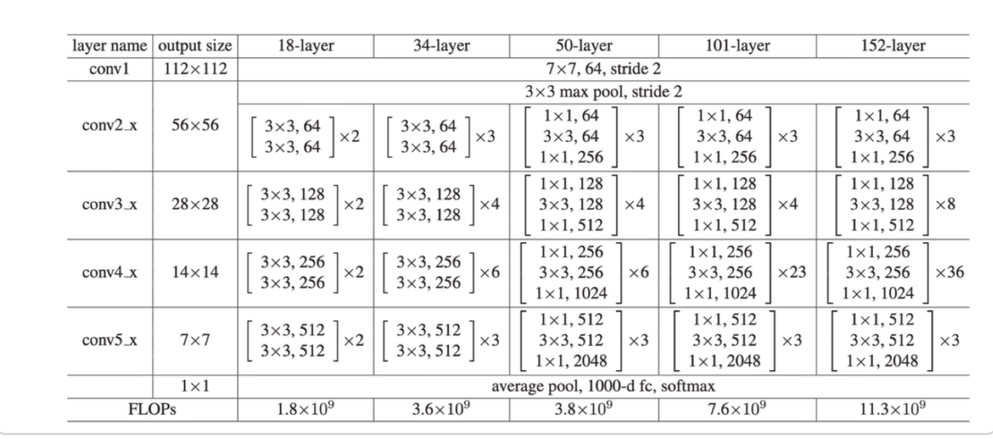
9/24 开始实现 ResNet-18 网络

基本思路：1. 先构建残差块，通过下图流程



输入
3×3卷积
批量归一化
ReLU
3×3卷积
批量归一化
对齐输入的维度和通道数
叠加
ReLU
输出结果

2. 将输出结果通过层层卷积，形成残差网络

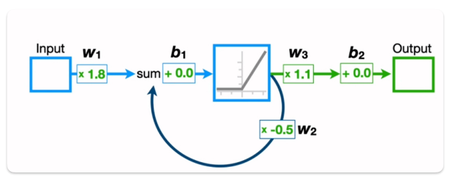


layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		[3×3, 64 3×3, 64] ×2	[3×3, 64 3×3, 64] ×3	[1×1, 64 3×3, 64 1×1, 256] ×3	[1×1, 64 3×3, 64 1×1, 256] ×3	[1×1, 64 3×3, 64 1×1, 256] ×3
conv3_x	28×28	[3×3, 128 3×3, 128] ×2	[3×3, 128 3×3, 128] ×4	[1×1, 128 3×3, 128 1×1, 512] ×4	[1×1, 128 3×3, 128 1×1, 512] ×4	[1×1, 128 3×3, 128 1×1, 512] ×8
conv4_x	14×14	[3×3, 256 3×3, 256] ×2	[3×3, 256 3×3, 256] ×6	[1×1, 256 3×3, 256 1×1, 1024] ×6	[1×1, 256 3×3, 256 1×1, 1024] ×23	[1×1, 256 3×3, 256 1×1, 1024] ×36
conv5_x	7×7	[3×3, 512 3×3, 512] ×2	[3×3, 512 3×3, 512] ×3	[1×1, 512 3×3, 512 1×1, 2048] ×3	[1×1, 512 3×3, 512 1×1, 2048] ×3	[1×1, 512 3×3, 512 1×1, 2048] ×3
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

二、Transformer Encoder

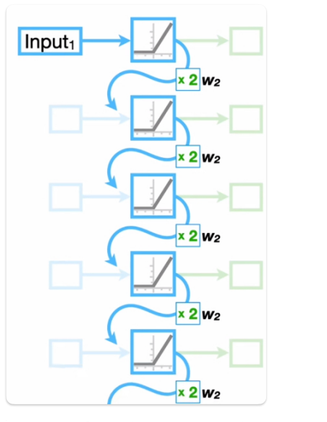
9/24 RNN循环神经网络

目的：使用不同数量的时间序列数据实现预测



Input
w1
x 1.8
sum
b1
+ 0.0
w3
x 1.1
b2
+ 0.0
Output
x -0.5
w2

劣势：展开的循环神经网络越多，训练越困难，即梯度爆炸/爆炸问题



Input1
x 2
w2

如上图所示，由于权重保持不变，循环次数增加后，数据将呈指数级增长，会产生梯度爆炸问题。若权重小于1，循环次数增加后，也会导致梯度消失问题

9/24～25 LSTM（长短期记忆神经网络）

针对RNN产生的梯度爆炸/消失的问题，使用LSTM来解决

通过和不同的反馈环连接，使用两条独立的路径，一条用于长期记忆，另一条用于短期记忆

长短期记忆使用Sigmoid激活函数和Tanh激活函数

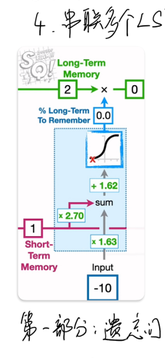
$f(x)=\frac{e^x}{e^x+1}\in(0,1)$　　$f(x)=\frac{e^x-e^{-x}}{e^x+e^{-x}}\in(-1,1)$

在LSTM中，记忆被分为2个部分：长期记忆与短期记忆

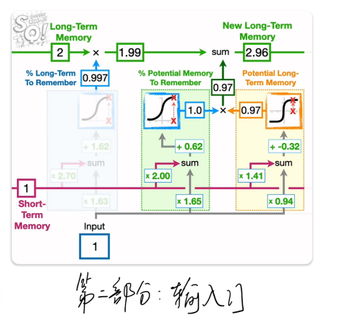
- 长期记忆：不能修改权重，独立于短期记忆，来避免梯度爆炸
- 短期记忆：可以调整参数，并且使用激活函数来确定LSTM需要保留的百分比。

步骤：

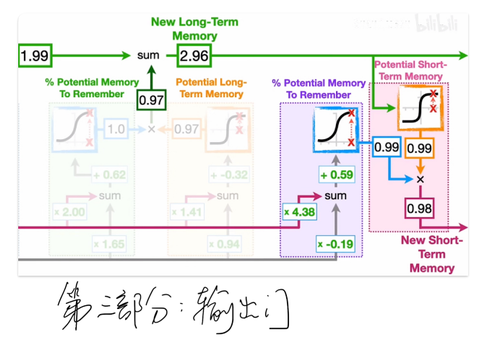
1. 考虑保留多少长期记忆
2. 由输入和短期记忆所计算的结果考虑有多少要成为新的长期记忆
3. 考虑有多少更新后的长期记忆成为下一个LSTM单元的短期记忆.
4. 串联多个LSTM单元



第一部分：遗忘门



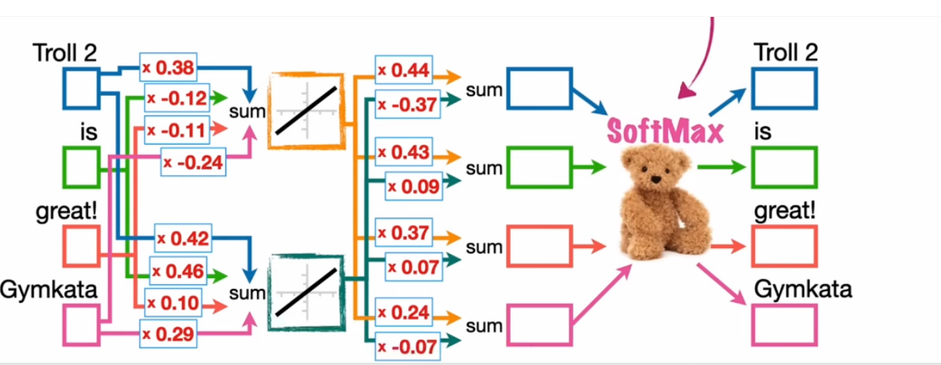
第二部分：输入门



第三部分：输出门

9/25 词嵌入（word embeddings）

背景：由于许多机器学习算法无法识别文字，所以当我们需要向神经网络中输入文字时，需要将文字转为算法可理解的数字



Troll 2
is
great!
Gymkata
x 0.38
x -0.12
x -0.11
x -0.24
x 0.42
x 0.46
x 0.10
x 0.29
sum
x 0.44
x -0.37
x 0.43
x 0.09
x 0.37
x 0.07
x 0.24
x -0.07
SoftMax
Troll 2
is
great!
Gymkata

方法一：连续词袋，通过使用周边的词来预测中间发生的词来增加语境。

方法二：跳跃模型，通过使用中间的词来预测周围的词来增加语境

word2vec 模型中，通常使用100个以上的激活函数和整个所连的节点进行训练，

也就是说，训练量十分庞大。

因此，需要考虑怎么加速word2vec，通过使用负采样，避免大量运算。

随机选择一部分单词来工作，不去预测用于优化的词。比方说，用单词A预测单词B，那么将除了A以外的单词的输入权重全部设为0，并且将除B以外的单词的输出权重全部设为0，达到加速的效果。

9/26 Transformer 模型

第一步：Word Embeddings

将词汇以词嵌入的方式，通过简单的神经网络，将词汇转化为数字组，创建嵌入值

（神经网络中的所有权重都通过反向传播来确定）

接着将嵌入值添加一组对应词序的数字，这些数字来自一系列交替的正、余弦函数，

最后将这些数字与开始的嵌入值相加，从而得到了每一个词汇的编码，也得到了整个句子的位置编码（训练时）

采用正弦函数可以通过调整周期大小来避免整个数组完全重复

如果仅仅只关注词汇本身的编码，可能会导致两句所含词汇完全相同的句子的混淆，所以为每句句子创建独有的位置编码

第二步：词与词之间的追踪，使用self-attention，完成编码器

在上一步中，已经完成了对于词汇的编码，首先对已经完成的编码再次放入简单网络进行数学运算，得到一对新的编码，称为Query。

接着，再为该词汇和句子中的下一个词创建两组新值，用于与Query计算相似度，其中创建这组新值时，经过的简单网络内的权重是完全一致的，这两组新值被称为Key。通过计算点积，并将两组点积结果通过SoftMax函数，来衡量词汇之间的相似度。同时，SoftMax的输出也可以看作为每个输入词用多少的比例编码之一。

在确定好比例后，再利用最初的词汇的位置编码，创建一组新值，称为value，并将SoftMax所创建的比例作为Value的缩放比例，再将缩放后的值进行组合的总和，就是词汇的自注意值（self attention values）

同理，每组词汇间都可以利用以上步骤计算自注意值（注意：无论Transformer中有多少不同的词汇，只需为query、key和value重复使用同一组权重）

最后，使用残差连接，将自注意值与位置编码相加。

那为什么要Q、V、K去算注意值呢，直接用原始的词汇位置编码表示词汇不行吗？

首先，每个词的新注意值包括其他词的输入，有助于为每个词提供上下文，有助于建立输出中每个词之间的关系。

其次，如果将每组计算Q、V、K的权重的单元看作一个自注意力单元，能处理复杂句中词汇中的关系，可以创建一系列自注意力单元，每个单元都有它自己的权重集，来捕捉词汇间的不同关系，这也称为多头注意力（Multi-head attention）

第三步：创建解码器

解码器以EOS标记的嵌入值开始，用EOS标记来启动解码

其余步骤与编码器相同，不再赘述。

第四步：连接编码器与解码器

以编码器中数据创建Key，以解码器的数据创建Query，再通过点积，SoftMax，与新建value相乘，再组合相加，得到Encoder-Decoder Attention，并再次添加残差连接，使其专心处理输入单词与输出单词间的关系而不用保留之间的自注意值和位置编码

第五步：最后的处理

将残差连接后得到的数据通过全连接层，再通过SoftMax，选择概率最大的词汇完成预测。

这样从一个词到下一个词的预测就算完成了，预测会持续直到最后输出EOS，完成解码。